Docker - Dockerfile

Un Dockerfile es un documento de texto plano que contiene todas las **instrucciones y comandos ordenados** necesarios para ensamblar una imagen de Docker de forma automatizada.

- **El plano frente al edificio:** Si la *imagen* de Docker es el edificio construido y listo para habitar, el Dockerfile es el plano arquitectónico con el que se construye.
- **Reproducibilidad:** Permite que cualquier alumno o servidor compile exactamente el mismo entorno de software sin depender del sistema operativo de su ordenador local.

Instrucciones Esenciales de un Dockerfile

Para explicar los fundamentos a los alumnos, conviene desglosar las directivas más utilizadas en el día a día profesional:

Instrucción	Descripción Pedagógica	Ejemplo Práctico
FROM	Define la imagen base (sistema operativo o entorno de ejecución inicial). Siempre es la primera línea.	FROM node:20-alpine
WORKDIR	Establece el directorio de trabajo dentro del contenedor para los comandos posteriores.	WORKDIR /app
COPY	Copia archivos o carpetas desde la máquina local (host) al interior del sistema de ficheros del contenedor.	COPY . .
RUN	Ejecuta comandos durante el proceso de **construcción** de la imagen (ej. instalar dependencias).	RUN npm install
EXPOSE	Documenta el puerto de red por el que el contenedor escuchará peticiones (informativo).	EXPOSE 3000
CMD	Define el comando por defecto que se ejecutará **al arrancar** el contenedor.	CMD ["npm", "start"]

Ejemplo Práctico 1: Dockerfile para un Servidor Web Estático (Nginx)

Este ejemplo es ideal para introducir el empaquetado de páginas web estáticas (HTML/CSS/JS).

Estructura del Proyecto

mi-proyecto-nginx/

├── Dockerfile

└── html/

  └── index.html

Contenido de los Archivos

- **html/index.html**:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Clase DAW - Docker</title>
</head>
<body>
    <h1>¡Hola desde mi contenedor con Nginx!</h1>
    <p>Desplegado paso a paso en el módulo de Despliegue.</p>
</body>
</html>
```

Dockerfile

```
# 1. Partimos de una imagen oficial ligera basada en Alpine Linux
FROM nginx:alpine

# 2. Copiar los archivos web locales a la ruta donde Nginx sirve contenido estático
COPY ./html /usr/share/nginx/html

# 3. Indicar el puerto de red estándar de HTTP
EXPOSE 80

# 4. Forzar a Nginx a ejecutarse en primer plano (obligatorio para contenedores)
CMD ["nginx", "-g", "daemon off;"]
```

Comandos de Compilación y Ejecución

Bash

```
# 1. Construir la imagen asignándole un tag
docker build -t web-nginx-daw .

# 2. Ejecutar el contenedor mapeando el puerto 8080 del host al 80 del contenedor
docker run -d -p 8080:80 --name servidor-estatico web-nginx-daw
```

Comprobación: Accede en el navegador a http://localhost:8080.

Ejemplo Práctico 2: Dockerfile para una Aplicación Backend (Node.js)

Ideal para cuando los alumnos empiezan a trabajar con servidores lógicos e instalación de dependencias de terceros.

Estructura del Proyecto

mi-backend-node/

├── Dockerfile

├── package.json

└── app.js

Contenido de los Archivos

package.json

```
{
  "name": "app-node-daw",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

app.js

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('¡Servidor Express funcionando dentro de Docker!');
});
```

```
app.listen(port, () => {
  console.log(`Servidor escuchando en el puerto ${port}`);
});
```

Dockerfile

```
# 1. Imagen oficial de Node.js optimizada con Alpine
FROM node:20-alpine

# 2. Directorio de trabajo dentro del contenedor
WORKDIR /app

# 3. Copiar primero los ficheros de dependencias para aprovechar la caché de Docker
COPY package*.json ./

# 4. Instalar las dependencias del proyecto
RUN npm install

# 5. Copiar el resto del código fuente
COPY . .

# 6. Exponer el puerto de la aplicación Node
EXPOSE 3000

# 7. Comando de inicio del servidor
CMD ["npm", "start"]
```

Práctica 0: Creación y despliegue de un Dockerfile personalizado con Nginx y persistencia.

- **Objetivo:** Comprobar cómo al modificar los archivos en el host los cambios se reflejan en los contenedores que usan el volumen asociado.
- **Tareas para el alumno:**
    1. Crear una página web HTML con su nombre y el título del ciclo formativo.
    2. Escribir el Dockerfile utilizando nginx:alpine.

3. Construir la imagen con el comando docker build. La imagen tendrá el nombre app-backend-daw
4. Levantar el contenedor mapeando un puerto libre del host con el puerto EXPOSE del servidor (ej: -p 9080:3000). El contenedor tendrá el nombre con-app-backend-daw
5. **Pregunta de reflexión:** ¿Qué ocurre si modificamos el archivo index.html en nuestra máquina local después de haber arrancado el contenedor? ¿Se actualiza automáticamente en el navegador? *(Introduce aquí el debate sobre la necesidad de los volúmenes o el uso de rebuild).*

Práctica 1: Creación y despliegue de un Dockerfile personalizado con MySQL y persistencia.

- **Objetivo:** Crear y desplegar una base de datos MySQL sencilla controlando como al modificar los datos se guardan en el volumen asociado.
- **Tareas para el alumno:**
  1. Crear la estructura del proyecto en la carpeta **practica-mysql-docker**. El volumen se montará en la carpeta **sql** del proyecto.
  2. Crear el Script de Inicialización **SQL** que cree al menos una tabla en la base de datos.
  3. Crear el **Dockerfile** Personalizado.
  4. Construir la imagen con el comando **docker build**. La imagen tendrá el nombre **mysql-personalizado-daw**
  5. **Ejecutar** el Contenedor con **Persistencia** mapeando el puerto 3306 del host con el puerto EXPOSE del servidor (ej: -p 3306:3306). El contenedor tendrá el nombre **contenedor-mysql-daw**. La variable MYSQL_ROOT_PASSWORD tiene el valor **root1234** y la variable MYSQL_DATABASE el valor **tienda_db**
  6. Ejecutar una **consulta** de **selección** contra la base de datos
  7. Ejecutar una **consulta** que **modifique** el contenido de la tabla productos.
  8. Reiniciar el contenedor y volver a ejecutar la consulta de selección para **comprobar** la **persistencia**
- **Documentación a entregar.** Se debe entregar un documento de nombre **Practica1 - TuNombre** que incluya la explicación y las capturas de pantalla que permitan verificar todas las tareas que se tienen que realizar.